
From a traditional to an AI-native SDLC

Issues we saw in our first weeks

Belief

Some distrust the output entirely, others expect miracles. It is the same mistake in opposite directions.

Capability

No training, too little access to AI tooling, and processes built for a world where writing code was the expensive part.

Foundation

Product knowledge lives in people's heads. AI workflows are fragmented, each only as good as whoever built it.

Adoption

The spread between teams is huge: some work almost AI-first, others barely use AI. There is no common way of working to close that gap.

The hardest constraint is not technical

"We've always done it this way"

Answered by someone inside the team who shows, on real work, that it can be done differently.

"I don't trust it. My code is better"

Answered by real changes that show the output meets your standards, once the context and the gates are set up well.

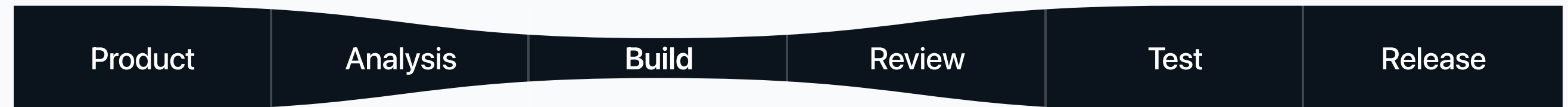
"It only creates slop"

Answered by gates. Slop is what unchecked output looks like. Most teams have review and CI, but not every team applies them consistently.

The bottleneck moves. It does not disappear.

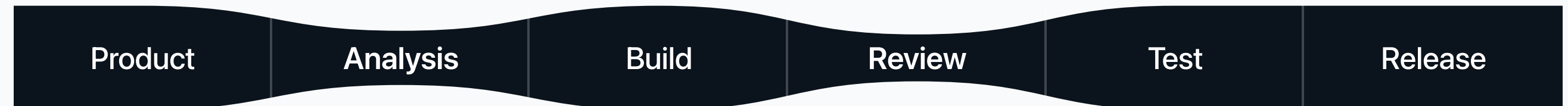
Today

Six desks, five handoffs



AI-assisted

What you ask and who checks it



AI-native

One run, four decisions



Four gates 1 Intent accepted 2 Spec approved 3 Change approved 4 Release authorised

Each row narrows where work waits: Build today, then Analysis and Review, then only the release sign-off.

At each gate a person decides, and the pipeline makes sure no gate is skipped.

Adapted from Anthropic's AI-Native SDLC playbook.

The AI-assisted stage is where people change

This is where a team learns the new way of working while the stakes are still low.

Learning in phases

Nobody goes from writing every line to reviewing an agent's work overnight. This is the step in between.

Trusting the output

Trust comes from working alongside the agent on real code and seeing its output hold up.

Living with gates

When an agent writes most of the code, the quality gates are what make it safe to ship.

What we propose, in this order

The goal is to do more with the same people.

01

Quality gates

Some teams already have strong gates, others far less. Every team reaches the same bar in CI/CD: today's standards, or stricter.

02

Platform

One way of working, owned by a small platform team and carried by the tooling instead of remembered by people.

03

Training

Teams learn to work with agents rather than around them.

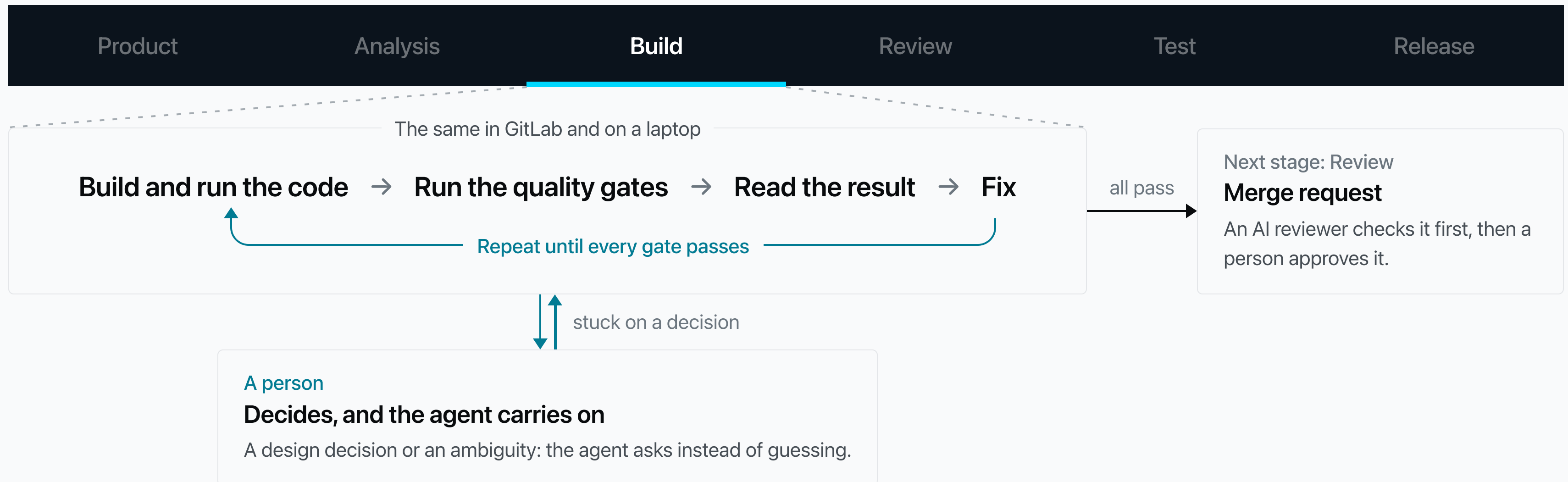
Access without gates buys you more code and less confidence in it.

What the quality gates check

Team standards	Code standards, agent guidelines and architecture rules per team.
Correctness	Unit and integration tests, a coverage threshold, evals in CI, API contract and migration checks, performance budgets and accessibility checks.
Security and compliance	Security, dependency and secret scans, no personal data in logs, and four-eyes approval with an audit trail.
Review	An AI reviewer checks acceptance criteria, guidelines, bugs, vulnerabilities and performance. Then a person reviews the merge request.

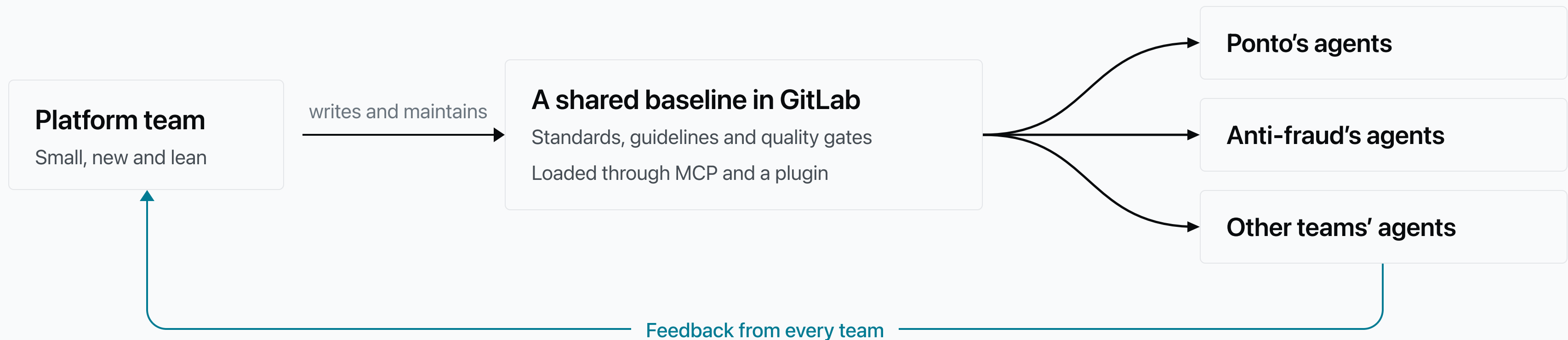
Through MCP the agent reads the ticket, and checks the running app against the spec.

Inside Build: the agent runs the gates itself



The gates hold every change to today's standards, or stricter, before a person ever sees it.

Platform engineering



A baseline, not a limit: each team still configures and extends its own agents on top of it.

"Have you heard of MCP?"

Asked in earnest, by developers already using AI, about a basic every developer should know by now. We also saw the agent used as a search box, and for small tasks instead of whole workflows.

The gap is not willingness. People do not yet know how to use AI well. Training closes that gap.

How we roll it out

Phase 1

Quality gates

Ponto and anti-fraud as pilot teams. Their gates hold AI output to today's standards, or stricter.

Phase 2

Platform

A small, new platform team codifies what worked in the pilots and ships it to every team.

Phase 3

Training

The other teams move onto the platform and learn to work with agents rather than around them.

What we need: a mandate across teams

Priority

Platform engineering as the primary track, with Ponto and anti-fraud as pilot teams. Decided once, so it is not reopened team by team.

Resources

More Izzy credits, weekly budgets or session limits, or a Claude subscription. Plus service accounts so agents run on remote machines.

People

A small, new platform team that owns the platform, and evangelists in the teams whose job is coaching and changing minds.

Feedback

A standing loop back to us, so we act on what the teams actually run into, not on what we assumed.

Thank you

Questions or concerns?

smartagents.be